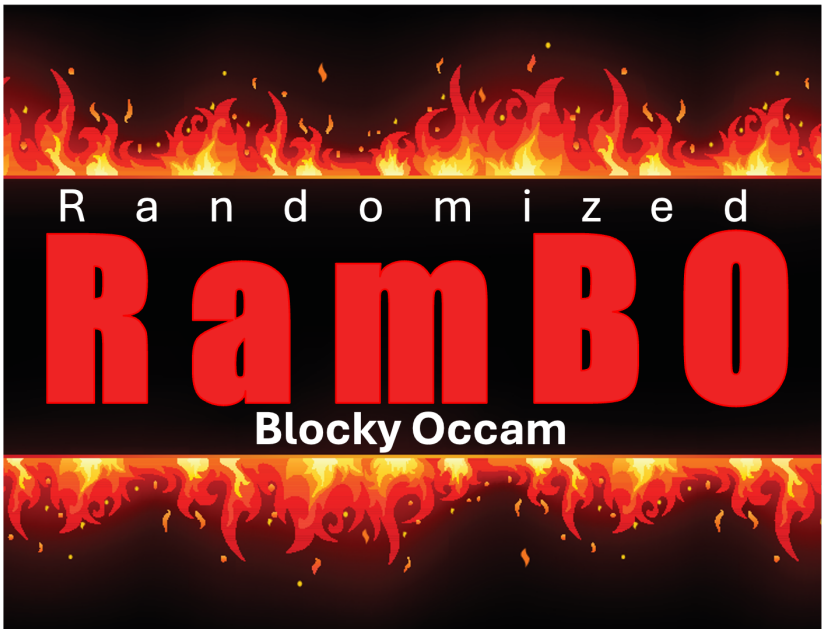
Randomized Blocky Occam

Open-source Matlab code for generating blocky models and uncertainties



Eliana Vargas Huitzil, Matthias Morzfeld, Steven Constable

Institute of Geophysics and Planetary Physics
Scripps Institution of Oceanography
University of California, San Diego

https://github.com/evargashuitzil/RamBO
https://marineemlab.ucsd.edu/Projects/RamBO/index.html

1 Overview

This document is the user guide for the Matlab implementation of RamBO, an open-source inversion program to generate blocky models and associated uncertainties. The code is based on the algorithms described in Vargas Huitzil et al. (2024). Please send bugs, questions, and/or suggestions to: evargashuitzil@ucsd.edu. The RamBO code is maintained and updated on a GitHub repository. You can access the repository at this link: https://github.com/evargashuitzil/RamBO.git To clone RamBO from GitHub, open a terminal and navigate to or create the directory where you want to store RamBO. Then type `git clone https://github.com/evargashuitzil/RamBO.git`.

1.1 How to cite this code

This code is part of the paper Vargas Huitzil et al. (2024). If the code is used in research or adapted for research, please cite our paper:

Vargas Huitzil, E., Morzfeld, M., and Constable, S. (2024). Rambo: Randomized blocky Occam, a practical algorithm for generating blocky models and associated uncertainties. Geophysical Journal International (2025).

1.2 License

1.3 Acknowledgments

This work was supported by NSF grant EAR-2433476 and by ONR grant N00014-21-1-2309. The work was done at the Institute of Geophysics and Planetary Physics, Scripps Institution of Oceanography, University of California, San Diego.

2 RamBO basics

RamBO works with optimizing the cost function

$$\mathcal{C}(m) = \|W(\mathcal{F}(m) - d)\|^2 + \mu \, |Dm| \, .$$

Here, m is an n_m-dimensional vector that represents the model we are looking for. The data d are a n_d-dimensional vector. The forward model $\mathcal{F}$ maps n_m-dimensional models to n_d dimensional data. W is a $n_m \times n_m$ diagonal matrix whose diagonal elements are the variances of the data errors, i.e., $W_{ii} = \sigma_{ii}^2$. The $n_m \times n_m$ matrix D implements a derivative and the scalar μ defines the strength of the regularization.

RamBO proceeds in two steps. First, the cost function is minimized via an Occam-type optimization during which the regularization strength is adjusted automatically. We call this step "blocky Occam." In a second step, RamBO computes an uncertainty quantification (UQ) by optimizing perturbed cost functions. The optimizations require the Jacobian of the forward model

$$J(m) = \frac{\partial \mathcal{F}}{\partial m},$$

which is a $n_d \times n_m$ matrix.

RamBO is agnostic to the forward model, which requires that the user specifies

1. the data d and associated standard deviations σ_i (Section 5);
2. a function that implements the forward model $\mathcal{F}$ and its Jacobian J (Section 5);
3. a function that implements the finite differencing matrix for the model (Section 6);
4. the starting model – RamBO is an iterative algorithm.

The user further needs to specify a few parameters of RamBO, in particular a target root mean square error (RMS) that controls the data fit and a range of possible regularization parameters μ.

3 Code structure

We recommend to download the code and store it in one central folder. The file `RunMe.m` contains an example of how to configure RamBO and how to provide the forward model and it's Jacobian on an example of a marine MT data set (Gustafson et al., 2019). The configuration and how to set up RamBO is described in Section 4.

Because RamBO is agnostic to the forward model, the user needs to supply the model and the data. This is done via files contained in the folder `UserSpecifiedFunctions`. Crucially, the functions `F.m` and `getD.m` are required. `F.m` implements the forward model and its Jacobian and expects certain input parameters (Section 5). `getD.m` implements the finite difference matrix and also expects certain input parameters (Section 6). `F.m` and `getD.m` are called by RamBO. If the code breaks, it is likely that `F.m` and `getD.m` are not implemented correctly.

The subfolder `RamBO_CoreFunctions` contains the RamBO code and need not be edited (a list of functions and short descriptions are provided in Section 9). Some plotting routines are provided in the folder `SummaryPlotting`. These files need to be adjusted based on the forward model.

4 Configuring and running RamBO

The file `RunMe.m` contains an example about how to configure and set up RamBO. First, we define the folder structure:

```
addpath('RamBO_CoreFunctions/') % Path to RamBO Code
addpath('UserSpecifiedFunctions/')% Path to user specified functions;
addpath('SummaryPlotting/')     % Path to plotting and summary routines
```

If you rename the folders, you need to adjust the path commands.

Next, we set up the model and the data.

```
%% Get model and data
%% ------------------------------------------------------------------------
ModelConfig = MakeModel;
DataConfig = MakeData;
%% ------------------------------------------------------------------------
```

We describe these steps in detail in Section 5. The two functions create the structures `ModelConfig` and `DataConfig`, which are used by RamBO to run the forward model (and compute its Jacobian). RamBO is configured by creating the structure `RamBOConfig`:

```
%% Configure & run RamBO
%% ------------------------------------------------------------------------
% Configure RamBO
RamBOConfig.TargetRMS       = 1;
RamBOConfig.muRange         = -3:.1:1;
RamBOConfig.MaxIts          = 20;
RamBOConfig.RoughTol        = 1e-2;
RamBOConfig.BregmanTol      = 1e-4;
RamBOConfig.BregmanMaxIts   = 300;
RamBOConfig.nos             = 50;
RamBOConfig.mu              = 0.1;
RamBOConfig.alpha           = 0.2;
RamBOConfig.RMSThreshold    = 3;
% Run RamBO
RamBO_Out = RamBO(RamBOConfig, ModelConfig, DataConfig);
%% ------------------------------------------------------------------------
```

The target RMS defines the desired model-data misfit. One, or slightly larger is a save choice. The `muRange` defines the values of the regularization parameter used during blocky Occam. `muRange` operates in log-space. `MaxIts` defines the maximum number of iterations during blocky Occam. `RoughTol` defines a tolerance on the model roughness; if relative changes in model roughness are below the tolerance, the optimization is stopped (and converged). `BregmanTol` defines a tolerance for convergence of split Bregman, which is used for solving linearized total-variation (TV) regularized inverse problems during the iterations. `BregmanMaxIts` defines the maximum number of iterations during split Bregman. Consult Vargas Huitzil et al. (2024) for details.

The remaining parameters are needed only for UQ. `nos` defines the number of samples for RamBO. Often, 50 may be sufficient to get a good idea of a UQ. If no UQ is needed, simply set `nos` to zero. `mu` is the regularization parameter in RamBO. A small number is a good choice to get maximum uncertainty. It is recommended to chose the smallest `mu` for which RamBO can converge (not encounter too many failed attempts at the optimization). `alpha` is a step-size for stability. If many

optimization attempts fail, decrease `alpha` or increase `mu`. `RMSThreshold` defines the minimum RMS for which we deem that the optimization has failed. Here, 2 or 3 are safe choices.

With the structures `RamBOConfig`, `ModelConfig` and `DataConfig`, we can run RamBO:

```
% Run RamBO
RamBO_Out = RamBO(RamBOConfig , ModelConfig , DataConfig);
```

The code produces several outputs, contained within a structure.

```
Blocky Occam Outputs
RamBO_Out.BO_modelEst      - Blocky Occam model in each iteration
RamBO_Out.BO_predRes       - Response in each iteration
RamBO_Out.BO_muAll         - Regularization strengths (mu) in each iteration
RamBO_Out.BO_RMS           - Root Mean Square (RMS) error in each iteration
```

These outputs summarize the iteration of blocky Occam. Provided are the models at each iteration of blocky Occam, the corresponding model outputs, regularization parameters and RMS values.

```
UQ
RamBO_Out.RamBO_modelEst   - Posterior samples
RamBO_Out.RamBO_predRes    - Responses of posterior samples
RamBO_Out.RamBO_RMS        - RMS of posterior samples
RamBO_Out.RamBO_nos        - Number of posterior samples
```

The above outputs contain the UQ. RamBO generates `nos` posterior samples (`modelEst`) and corresponding model outputs (`predRes`) and RMS values. The outputs and their interpretation are model dependent, but we provide an example of how to illustrate the results with figures and output summaries in Section 7.

5 Configuring the grid, data and forward model

We provide an example of how to provide RamBO with a forward model and data. Within the `RunMe.m` script, the user needs to provide the functions `MakeModel` and `MakeData`.

`MakeModel` configures the model and generates a structure called `ModelConfig`. This structure contains everything one would need to run the forward model. Crucially, `ModelConfig` also must contain the starting model – RamBO is an iterative algorithm that needs a starting point. The example sets up a simple, uniform grid ranging from 20m to 1200m depth with a layer thickness of 20m. The starting model is a half-space.

```
%% 1D grid (uniform)
depth = 1200;                                          % depth
ModelConfig.lt = 20;                                   % layer thickness
ModelConfig.z = ModelConfig.lt:ModelConfig.lt:depth;   % grid
ModelConfig.h = ModelConfig.lt*ones(ModelConfig.nm,1); % helper vector
%% Required: the initial model and number of unknowns/layers
ModelConfig.mo = 2*ones(ModelConfig.nm,1);             % initial model
ModelConfig.nm = length(ModelConfig.z);                % number of layers
```

`ModelConfig.nm` is required and specifies the number of unknowns (layers in our example).

`MakeData` loads and provides the data and associated standard deviations (not variances). In our example, the data are apparent resistivities (log-space) and phases at given frequencies.

```
load('MTdata.mat');                    % Load MT data and get it into a structure
DataConfig.d = [MTdata.TEappRes' ; MTdata.TEphase'];          % data, log apparent
    resistivity and phase
DataConfig.s = [MTdata.TEappResErr' ; MTdata.TEphaseErr'];    % errors, log domain
DataConfig.f = MTdata.freqs';                                  % frequencies
DataConfig.nd = length(DataConfig.d);                          % number of data
```

`DataConfig.nd` is required and specifies the number of data points.

The structures `ModelConfig` and `DataConfig` should contain all parameters that are needed to run the forward model and to compute its Jacobian. The forward model and Jacobian are implemented in the function `F.m`. The inputs are a model `m`, and the `ModelConfig` and `DataConfig` structures. This function is called by RamBO and needs to adhere to the input structure (`m`, `ModelConfig`, `DataConfig`). The example contains an MT forward code (`callMT.m`).

```
function [ModelOutput,J] = F(m,ModelConfig,DataConfig)

% This function implements the forward model
% RamBO expects the output to be a vector of the size of the data
% The second output is the Jacobian of the model

nd  = DataConfig.nd;
h   = ModelConfig.h;
f   = DataConfig.f;
ModelOutput = callMT(m,h,f);

% Jacobian
if nargout == 2
    J = Jac(m,h,f,nd,0.05);
end

end
```

The function `Jac` computes the Jacobian. In our example, the Jacobian is computed via finite differences using the forward code `callMT`, but analytical derivatives are just as good (or better). It is important that the function `F.m` is implemented to require only three inputs (model `m`, model parameters `ModelConfig` an data parameters `DataConfig`) because the RamBO code calls the function `F.m` with this expectation.

6 Finite differencing matrix

RamBO requires the function `getD`, which implements the finite differencing matrix. This matrix depends on the model and requires the structure `ModelConfig` as the only input parameter. In our example, we use a simple finite differencing on a 1D uniform grid.

```
function D = getD(ModelConfig)
%% Make the finite differencing matrix for regularization
nm = ModelConfig.nm;  % number of model parameters (nm)
% Finite differences for regularization
D = diag(ones(nm,1),0) - diag(ones(nm-1,1),-1); D(1,1) = 0;
```

7 Plotting

The folder `SummaryPlotting` provides some ideas of how to illustrate and summarize the results of RamBO, based largely on the figures in Vargas Huitzil et al. (2024). These functions need to be adjusted based on the forward model. It's a good idea to plot RMS during the iteration of blocky Occam, and to look at the blocky Occam model and associated data fits. For a UQ, we recommend plotting the various samples and their data misfit, as well as a histogram of RMS of the samples.

8 Occam's inversion

Our code also contains an implementation of Occam's inversion (Constable et al., 1987). Occam's inversion requires the same in terms of model, data and derivative matrix as RamBO. We can configure Occam's inversion using the structure `OccamConfig`:

```
OccamConfig.TargetRMS = 1;
OccamConfig.muRange   = -3:.1:1;
OccamConfig.MaxIts    = 30;
OccamConfig.tol       = 1e-2;
% Run Occam
Occam_Out = Occam(OccamConfig, ModelConfig, DataConfig);
```

The configuration defines the desired model-data misfit (RMS), the range of regularization parameters to be considered (log-space), the maximum number of iterations and a tolerance for convergence.

9 RamBO core functions

We list and provide short descriptions of the core functions of RamBO.

`RamBO`: Implemenation of RamBO

`blockyOccam`: Implementation of blocky Occam (variable regularization parameter).

`blockyOccamFixedMu`: Implementation of blocky Occam (fixed regularization parameter).

`Occam`: Implementation of Occam's inversion.

`BregmanSplit`: Implementation of split Bregman for linear TV-regularized inverse problems.

`soft_thresh`: Soft-thresholding (used in split Bregman).

`laprnd`: Draws Laplace distributed random variables (used for perturbing the cost function in RamBO).

`brewermap`: Nice colors for plotting.

References

Constable, S., Parker, R., and Constable, C. (1987). Occam's inversion: A practical algorithm for generating smooth models from electromagnetic sounding data. Geophysics, 52(3):289–300.

Gustafson, C., Key, K., and Evans, R. L. (2019). Aquifer systems extending far offshore on the U.S. Atlantic margin. Scientific Reports, 9(1):1–10.

Vargas Huitzil, E., Morzfeld, M., and Constable, S. (2024). RamBO: Randomized blocky Occam, a practical algorithm for generating blocky models and associated uncertainties. Geophysical Journal International.